DATE: 24-03-2026

TOPIC: SEMAPHORES (PRODUCER & CONSUMER)

OUTPUT:

```
Activities   Terminal                    Mar 24  11:02
                          maitreyee@localhost:~/sphore
-          bankers    mai      Music      priority.c    sh          whilw.sh
a          bm         maii.c   os_lab     Public        sjf         XYZ
ABC        close      mait     os_lab1    REC           sjf.c
abc.sh     Desktop    maithu   os_lab1.c  roundrobin    Templates
ab.sh      Documents  maitre   os_lab.c   roundrobin.c  test.sh
bank       Downloads  mash     Pictures   sample.c      Videos
banker.c   h          mithii   priority   sample.txt    while.sh
[maitreyee@localhost ~]$ mkdir sphore
[maitreyee@localhost ~]$ cd sphore
[maitreyee@localhost sphore]$ vi sphore.c
[maitreyee@localhost sphore]$ gcc sphore.c -o sphore
[maitreyee@localhost sphore]$ ./sphore
Produced: 0
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
[maitreyee@localhost sphore]$ vi sphore.c
[maitreyee@localhost sphore]$
```

Activities Terminal Mar 24 11:03

maitreyee@localhost:~/sphore — /usr/bin/vim sphore.c

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
sem_t mutex;

void *producer(void *arg)
{
    int item;
    for(int i = 0; i < BUFFER_SIZE; i++)
    {
        item = i;
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consumer(void *arg)
```

Activities Terminal Mar 24 11:04

maitreyee@localhost:~/sphore — /usr/bin/vim sphore.c

```c
void *consumer(void *arg)
{
    int item;

    for(int i = 0; i <BUFFER_SIZE; i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);
    }
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
```